

引入

动态规划也被称为dp，是一种利用已知信息，通过递推方程来实现对未知信息的获取。一般利用动态规划解决的问题，代码量都很简短，但是并不意味着它简单，恰恰相反，它对思维的要求很高，他也足够的抽象。

dp有很多的种类：线性dp，区间dp，数位dp，树形dp，状压dp等等。

最经典的dp问题——背包问题

背包问题又有很多种类：01背包、完全背包、分组背包、有依赖的背包、多重背包、混合背包等。

例题1.走楼梯

假设你正在走楼梯，还需要走 n 阶才能到楼顶。每次你可以走1阶楼梯或者2阶楼梯，请问你一共有多少种不同的方法到楼顶。

解法：

我们在解决动态规划问题最经常使用的思考方法是**考虑最后一步的决策**，在上面的问题中，很容易想到一阶台阶的方案数为1，两阶台阶的方案数为2，三阶台阶方案数为3，四阶以及四阶以上就不是很好想了，这里我们先考虑四阶台阶，**一个合法的方案的最后一步要么是走一阶台阶，要么是走两阶台阶，且这两种类型的方案数相互独立**，如果是最后一步走一阶台阶的情况，对于四阶台阶我们就先得走三阶，也就是四阶台阶的方案数至少有一部分是三阶台阶的方案数，如果是最后一步走两阶台阶的情况，对于四阶台阶我们就先得走两阶，也就是四阶台阶的方案数另外一部分是两阶台阶的方案数，即四阶台阶的方案数为

3 （三阶台阶方案数）+ 2 （两阶台阶方案数）= 5 ；

类比到 n 阶台阶，设 $f(n)$ 为 n 阶台阶方案数，得到递推方程为：

$$f(n) = f(n-1) + f(n-2)$$

这个递推方程就是经典的斐波那契数列。

代码：

```
#include<bits/stdc++.h>

using namespace std;

const int N=50;

int f[N]; //n阶台阶的方案数

int main()
```

```

{

    int n;cin>>n;

    f[1]=1,f[2]=2;

    for(int i=3;i<=n;i++)f[i]=f[i-1]+f[i-2];//递推方程

    cout<<f[n]<<'\n';

    return 0;

}

```

动态规划问题的求解步骤

- 1.首先确定状态表示;
- 2.确定状态的初始值;
- 3.确定状态转移方程。

(足够抽象吧, 确实。个人认为动态规划问题没有一套完整的求解步骤, 每一道题都有着每一道题特有的解法, 只有不断的写题, 在考场上才会有灵光乍现的想法)

以上三点为解决动态规划问题所必须确定的要素。

线性dp

递推方程是一个线性表达式

例题2. [最大字段和](#)

题意: 给出一个长度为 n 的序列 a , 选出其中连续且非空的一段使得这段和最大。

思路:

- 1.确定状态表示: 设 $f[i]$ 为以 $a[i]$ 元素结尾的最大子段和;
- 2.确定状态初始值: $f[1]=a[1]$;
- 3.确定状态转移方程: 假设我们来到了 i 位置, 我们现在要确定 $f[i]$, 因为 $f[i]$ 为以 $a[i]$ 元素结尾的最大子段和, 所以 $f[i]$ 至少有 $a[i]$, 我们需要确定 $f[i]$ 应不应该向前取数, 此时我们只需要判断 $f[i-1]$ 是不是大于0的数, 因为 $f[i-1]$ 它表示以 $a[i-1]$ 结尾的最大字段和, 所以 $f[i]$ 想要向前取数的话必定要加 $f[i-1]$

1], 但 $f[i-1]$ 如果是负数的话还不如不加, 所以最终的状态转移方程为

```
f[i]=max(f[i-1],0)+a[i];
```

最后从1到n找一遍 $f[i]$ 的最大值即可。

代码:

```
#include<bits/stdc++.h>

using namespace std;

const int N=2e5+10;

int a[N];

int f[N];

int main()

{

    int n;cin>>n;

    for(int i=1;i<=n;i++)cin>>a[i];

    for(int i=1;i<=n;i++)

    {

        f[i]=max(f[i-1],0)+a[i];

    }

    int ans=-1e9;

    for(int i=1;i<=n;i++)

    {

        ans=max(ans,f[i]);

    }

}
```

```
}

cout<<ans<<'\n';

return 0;

}
```

二维dp

在一个二维图上进行dp

例3.最小路径和

题意：给定一个包含非负整数的 $m \times n$ 网格 `grid`，请找出一条从左上角到右下角的路径，使得路径上的数字总和为最小。

说明：每次只能向下或者向右移动一步。

1	3	1
1	5	1
4	2	1

思路：

- 1.确定状态转移方程：设 $f[i][j]$ 为从 $(1, 1)$ 点到 (i, j) 点的最小路径和
- 2.确定状态初始值： $f[i][j] = a[i][j]$
- 3.确定状态转移方程：假设我们来到了 (i, j) 位置，我们只有可能从 $(i-1, j)$ 来到 (i, j) 或者从 $(i, j-1)$ 来到 (i, j) ，所以 $f[i][j] = \min(f[i-1][j], f[i][j-1]) + a[i][j]$ ，但是要注意如果 i 为1的话， (i, j) 只能从 $(i, j-1)$ 转移，同理，当 j 为1的话， (i, j) 只能从 $(i-1, j)$ 转移。

代码：

```
class Solution {
```

```

public:

    int minPathSum(vector<vector<int>>& grid) {

        int n=grid.size();

        int m=grid[0].size();

        int f[n+2][m+2]={1e9};

        for(int i=1;i<=n;i++)

        {

            for(int j=1;j<=m;j++)

            {

                if(i!=1)f[i][j]=min(f[i][j],f[i-1][j]);

                if(j!=1)f[i][j]=min(f[i][j],f[i][j-1]);

                f[i][j]+=a[i][j];

            }

        }

        return f[n][m];

    }

};

```

01背包

例4.采药

题意：有一个容量为V的背包，还有n个物体。现在忽略物体实际几何形状，我们认为只要背包的剩余容量大于等于物体体积，那就可以装进背包里。每个物体都有两个属性，即体积w和价值v。

问：如何向背包装物体才能使背包中物体的总价值最大？

$w[i]$ 表示第*i*件商品的体积， $v[i]$ 表示第*i*件商品的价值

思路：

1.确定状态表示： $f[i][j]$ 表示前*i*件商品，当背包体积为*j*时所能装的最大价值

2.确定状态初始值： $f[i][0] = 0$;

3.确定状态转移方程：

假设现在对第*i*件商品做决策，我们有两种情况：

第一种，不选第*i*件商品，则 $f[i][j] = f[i-1][j]$ ，表示不选第*i*件商品，最大价值应该为前*i-1*件商品的最大价值；

第二种，选第*i*件商品，则 $f[i][j] = f[i-1][j-w[i]] + v[i]$ ，表示选第*i*件商品，前*i-1*件商品，当背包体积为*j-w[i]*时的最大价值加上 $v[i]$ 。因为需要选第*i*件商品，所以应该给*i*件商品留出 $w[i]$ 的空间，所以*j*需要减去 $w[i]$ 。

综上，状态转移方程为：

$$f[i][j] = \max(f[i-1][j], f[i-1][j-w[i]] + v[i]);$$

代码：

```
#include<bits/stdc++.h>

using namespace std;

const int N=1010;

int f[N][N];

int w[N];

int v[N];

int main()

{

    int W,n;cin>>W>>n;

    for(int i=1;i<=n;i++)

    {
```

```
        cin>>w[i]>>v[i];

    }

    for(int i=1;i<=n;i++)

    {

        for(int j=1;j<=W;j++)

        {

            f[i][j]=f[i-1][j];

            if(j>=w[i])f[i][j]=max(f[i][j],f[i-1][j-w[i]]+v[i]);

        }

    }

    cout<<f[n][W]<<'\n';

    return 0;

}
```